

Debugging

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Software hat (immer) Fehler

- **Testen** = Aufdecken/Provozieren von Fehlern
- **Debuggen** = Finden der Stelle im Code

Warum Debuggen?

Typische Fehlerarten:

- **Exceptions:** Programm bricht ab
- **Logikfehler:** Programm läuft durch, Ergebnis ist aber falsch

Warum Debugger statt `IO.println`:

- **Alle Variablenwerte** in einem Zustand sichtbar
- Programm **Schritt für Schritt** ausführbar
- **Call Stack** (Wer hat wen aufgerufen?)
- Komplexe Bedingungen (z.B. **bedingte Breakpoints**)

Die wichtigsten Debugger-Werkzeuge

The screenshot displays the IntelliJ IDEA IDE interface. The top toolbar shows the 'Run' button (a green play icon) and the 'Debug' button (a green bug icon). The main editor window shows the source code of 'DebugDemo.java'. A breakpoint is set at line 29, which is highlighted in blue. A context menu is open over this breakpoint, offering options: 'Run \'DebugDemo.main()\'', 'Debug \'DebugDemo.main()\'', 'Profile \'DebugDemo.main()\' with \'IntelliJ Profiler\'', and 'Modify Run Configuration...'. The 'Debug' button in the menu is highlighted. The left sidebar shows the project structure, including 'debugging', 'gradle', 'build', 'src', 'main', 'java', and 'debugging'. The bottom panel shows the 'Debug' window with the 'Threads & Variables' tab active. It displays the state of the program during a debug session, showing the 'main' thread running and the 'numbers' variable as an 'ImmutableCollections\$ListN@1011' with a size of 5. The list contains the following elements:

- 0 = {Integer@1015} 2
- 1 = {Integer@1016} 4
- 2 = {Integer@1017} 6
- 3 = {Integer@1018} 8
- 4 = {Integer@1019} 10

Standard-Vorgehen beim Debuggen

1. Fehler beobachten

- Was genau passiert? Exception? Falsches Ergebnis?

2. Fehler reproduzierbar machen

- Konkrete Eingaben / Szenario festlegen

3. Hypothese bilden

- Was könnte im Code falsch laufen?

4. Breakpoint setzen

- In einer relevanten Methode / vor einer verdächtigen Stelle

5. Im Debug-Modus starten

- Programm läuft bis zum Breakpoint

6. Schrittweise ausführen & Variablen beobachten

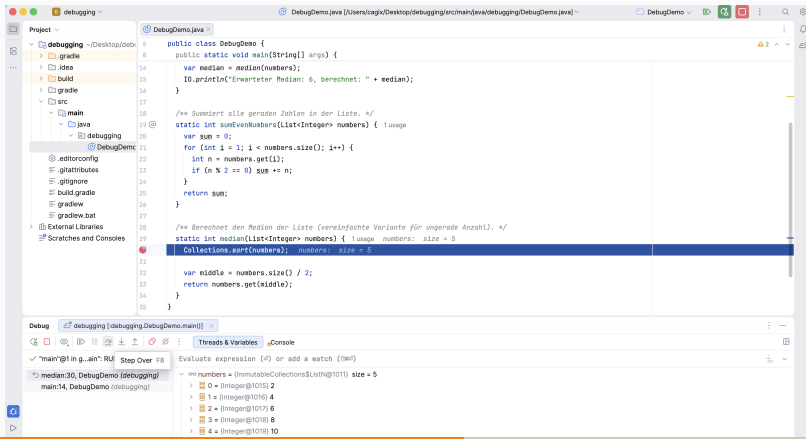
- `Step Over` / `Step Into`, Variablenfenster, ggf. Ausdrucksauswertung

7. Fehler lokalisieren und fixen

- Code korrigieren, dann wieder bei Schritt 1 beginnen

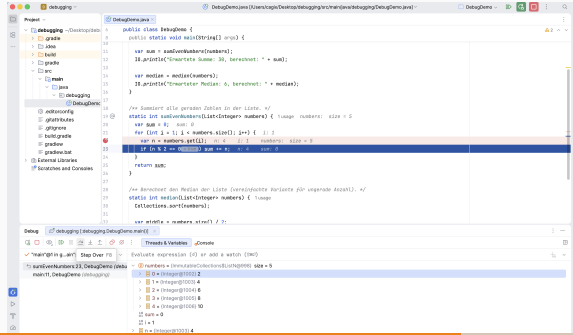
Demo 1: Exception beim Median (Crash)

```
static int median(List<Integer> numbers) {  
    Collections.sort(numbers); // BUG: UnsupportedOperationException  
    var middle = numbers.size() / 2;  
    return numbers.get(middle);  
}
```



Demo 2: Logikfehler bei der Summe (falsches Ergebnis)

```
static int sumEvenNumbers(List<Integer> numbers) {  
    var sum = 0;  
    // BUG: Schleife startet bei 1, Element mit Index 0 (Wert 2) wird nie besucht  
    for (int i = 1; i < numbers.size(); i++) {  
        var n = numbers.get(i); if (n % 2 == 0) sum += n;  
    }  
    return sum;  
}
```



Tipps zum selbstständigen Üben

- Probieren Sie in Ihrer IDE:
 - Einen **Bedingten Breakpoint**: z.B. in der Schleife von `sumEvenNumbers` nur halten, wenn `i == 0` oder `n > 5`
 - Das **Ändern von Variablenwerten** im Debugger
 - Das **Auswerten von Ausdrücken** (Evaluate Expression)
- Ersetzen Sie in `main` die Beispiel-Liste durch andere Daten:
 - z.B. leere Liste, eine Liste mit nur einem Element, ungerade/gerade Anzahl
- Ziel: Sie sollten das Gefühl haben, dass Sie den Debugger **aktiv steuern** können, statt nur “zuzuschauen”.

- Debugger ist zentrales Werkzeug:
 - Breakpoints, Step Over/Into/Out, Variablen, Call Stack
- Vorgehen:
 - Fehler beobachten -> reproduzieren -> Hypothese -> Breakpoint -> schrittweise ausführen -> fixen



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

